



POLITECHNIKA POZNAŃSKA

WYDZIAŁ INFORMATYKI I TELEKOMUNIKACJI
Instytut Informatyki

Praca dyplomowa magisterska

TEMAT PRACY DYPLOMOWEJ

Szymon Krzysztof Gogulski, 147403

Promotor
dr inż. Michał Melosik

POZNAŃ 2026

Tutaj będzie karta pracy dyplomowej;
oryginał wstawiamy do wersji dla archiwum PP, w pozostałych kopiach wstawiamy ksero.

Spis treści

1	Wstęp	1
1.1	Zapotrzebowanie na niezawodne źródła losowości	1
1.2	Cel pracy	1
1.3	Charakterystyka literatury	1
1.4	Układ pracy	2
2	Podstawy teoretyczne	3
2.1	Entropia Shannon’a	3
2.2	PRNG a TRNG	3
2.3	Oscylator pierścieniowy	4
2.4	Architektura modułu neoTRNG	4
2.5	Korekcja źródła	5
2.5.1	Korektor Von-Neumanna	5
2.6	Metody przetwarzania końcowego	5
2.6.1	XOR	5
2.6.2	CRC	5
2.6.3	Ekstraktor Toeplitz’a	5
2.6.4	AES-256	5
2.7	Metody badania źródeł entropii	5
2.7.1	ENT	5
2.7.2	NIST SP 800-22	5
2.7.3	NIST SP 800-90B	5
2.8	Moduły testowe	5
2.8.1	RCT	5
2.8.2	APT	5
2.9	Sprzęt	5
2.9.1	Digilent CoraZ7-07S	5
2.9.2	Digilent Nexys 4 DDR	5
3	Testy bazowe	6
3.1	Modyfikacja	6
3.2	NIST SP 800-22	6
4	Analiza rozmieszczenia	8
5	Faza algorytmiczna i optymalizacja ekstrakcji	9
6	Faza odporności i monitorowanie online	10

7	neoTRNG na FPGA dla RPI	11
8	Zakończenie	12
	Literatura	13
A	Kod źródłowy	14
A.1	neoTRNG_mod_1.vhd	14
B	Raport	21
C	Notatki	22

Spis rysunków

2.1	Model źródła entropii. Źródło: [5]	3
2.2	Przykład prostego oscylatora pierścieniowego.	4
2.3	Architektura neoTRNG. Źródło: [4]	4

Spis tabel

3.1	NIST SP 800-22 Statistical Test Results	7
-----	---	---

Rozdział 1

Wstęp

1.1 Zapotrzebowanie na niezawodne źródła losowości

Generatory liczb losowych stanowią podstawę algorytmów współczesnej kryptografii [3]. Generowanie par kluczy kryptograficznych oraz tworzenie podpisów cyfrowych nie byłyby możliwe bez odpowiednich źródeł losowości. Jednocześnie generatory liczb losowych, które nie spełniają wymagań statystycznych oraz miar losowości, mogą znacząco osłabiać bezpieczeństwo stosowanych algorytmów. Przesłanki te stanowią motywację do podjęcia badań nad oceną dostępnych generatorów.

1.2 Cel pracy

Celem niniejszej pracy magisterskiej jest implementacja oraz analiza generatorów losowości bazujących na oscylatorach pierścieniowych. Badane moduły będą implementowane jako układy cyfrowe realizowane na płytach deweloperskich wyposażonych w układy FPGA.

Kryteria oceny generatorów obejmować będą: zestaw testów statystycznych zdefiniowanych w publikacji NIST SP 800-22 [2], zestaw testów do oceny entropii opisany w publikacji NIST SP 800-90B [5] oraz program do oceny generatorów pseudolosowych ENT [1]. Wraz z wynikami testów zestawione zostaną informacje o zajętości zasobów sprzętowych poszczególnych implementacji. Badane moduły będą stanowić modyfikacje bazowego rozwiązania neoTRNG przedstawionego w repozytorium [4]. Modyfikacje obejmą aspekty takie jak: liczba zastosowanych oscylatorów, zastosowanie korekcji źródła (np. korektor Von Neumanna), metody przetwarzania końcowego (post-processing) oraz fizyczne rozmieszczenie elementów w strukturze FPGA.

Osobna faza badań poruszy kwestie analizy odporności układu oraz mechanizmów monitorowania. Zbadany zostanie wpływ modułów testowych (Health Tests) oraz wpływ warunków środowiskowych na jakość badanego źródła entropii.

Dodatkowy etap implementacyjny obejmie porównanie zastosowania neoTRNG jako generatora entropii dla komputera jednopłytkowego Raspberry Pi z rozwiązaniem komercyjnym, takim jak moduł Quantis.

1.3 Charakterystyka literatury

Badania zostaną przeprowadzone w oparciu o rekomendacje branżowe dotyczące projektowania źródeł entropii oraz publikacje naukowe związane z analizowaną tematyką.

1.4 Układ pracy

Struktura pracy obejmuje osiem rozdziałów oraz dodatkową sekcję załączników.

Rozdział pierwszy przedstawia motywację podjęcia badań w obszarze generatorów entropii oraz określa założenia i cele pracy.

Rozdział drugi koncentruje się na podstawach teoretycznych, omówione zostaną definicje związane z tematyką pracy, architektura badanego źródła entropii, a także metody korekcji źródła, metody przetwarzania końcowego oraz techniki oceny (testy statystyczne i moduły testowe).

Rozdział trzeci rozpoczyna część badawczą pracy i obejmuje analizę wpływu korekcji źródła oraz metod przetwarzania końcowego na poziom entropii i zajętość zasobów sprzętowych.

Rozdział czwarty poświęcony jest analizie wpływu fizycznego rozmieszczenia oscylatorów w strukturze układu na poziom entropii.

Rozdział piąty przedstawia porównanie różnych metod przetwarzania końcowego.

Rozdział szósty rozszerza zakres badań o zagadnienia związane z monitorowaniem działania źródła oraz wpływem warunków środowiskowych.

Rozdział siódmy omawia dodatkowe aspekty implementacyjne oraz zawiera porównanie modułu wykorzystującego oscylatory pierścieniowe z komercyjnym generatorem Quantis.

Rozdział ósmy kończy pracę i podsumowuje najważniejsze wnioski wynikające z przeprowadzonych badań.

Rozdział 2

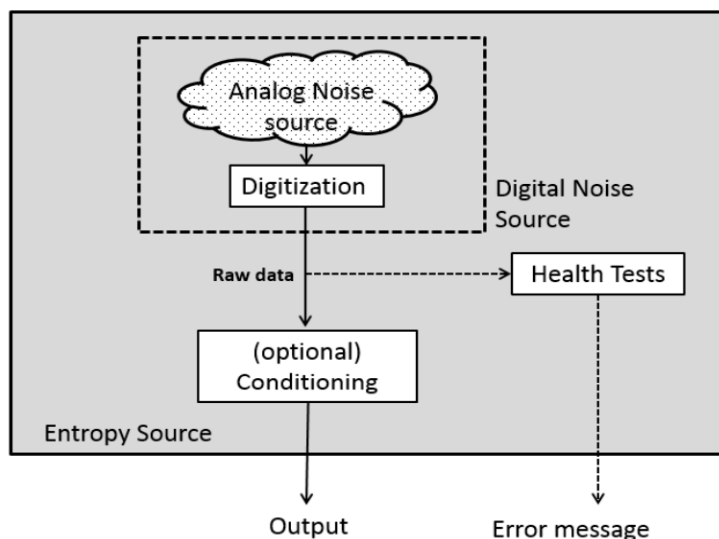
Podstawy teoretyczne

Projektowanie sprzętowych generatorów liczb losowych wiąże się z syntezą wiedzy z wielu dziedzin: elektroniki cyfrowej, fizyki, przetwarzania sygnałów, kryptografii oraz teorii informacji. W niniejszej części przedstawiono najważniejsze pojęcia związane z tematyką pracy. Wyjaśnione zostaną zarówno pojęcia fundamentalne jak losowość według Shannon'a oraz bardziej złożone zagadnienia architektury źródła entropii.

2.1 Entropia Shannon'a

1. Wstaw podręcznikową definicję, pamiętaj o \cite
2. porównaj dwa sygnały, 010101010101010101, i 11001011101110100011 za pomocą ENT

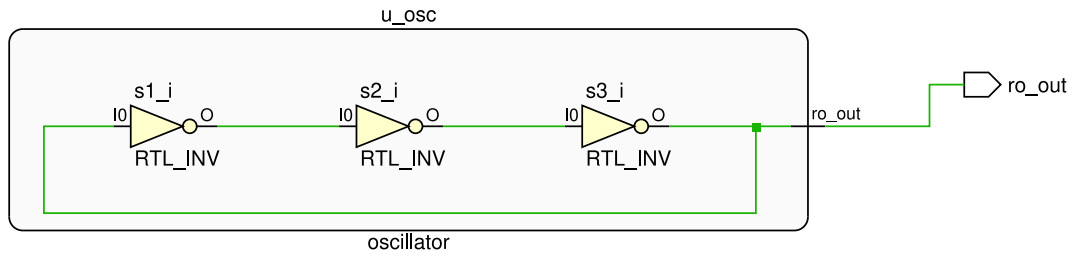
2.2 PRNG a TRNG



RYSUNEK 2.1: Model źródła entropii. Źródło: [5]

1. Omów różnicę między PRNG i TRNG, najlepiej na podstawie źródła
2. Podkreśl znaczenie pochodzenia szumu ze źródła analogowego
3. Opisz model źródła nist

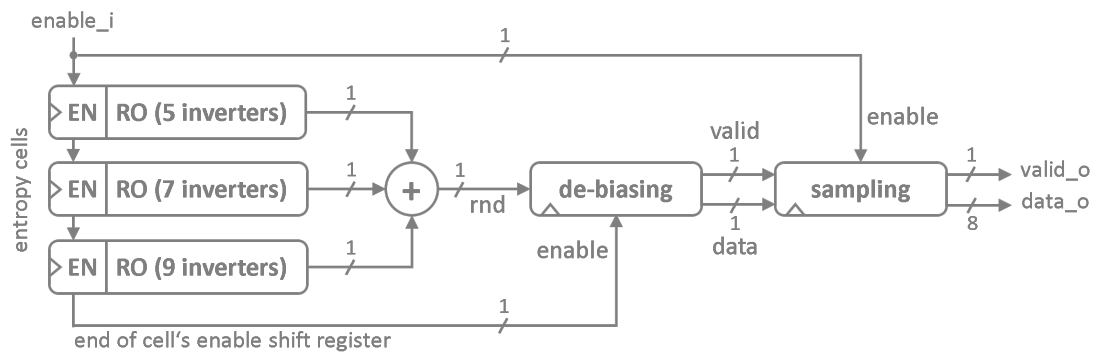
2.3 Oscylator pierścieniowy



RYSUNEK 2.2: Przykład prostego oscylatora pierścieniowego.

1. Opisz oscylator pierścieniowy/ (pętlę kombinatoryczną z bramką NOT)
2. przedstaw najprostszy przykład: RTL, układ, symulację, widok z oscyloskopu.
3. sprawdź czy tak prosty przykład będzie generować sygnał losowy

2.4 Architektura modułu neoTRNG



RYSUNEK 2.3: Architektura neoTRNG. Źródło: [4]

1. Opisz elementy składowe modułu

2.5 Korekcja źródła

2.5.1 Korektor Von-Neumanna

2.6 Metody przetwarzania końcowego

2.6.1 XOR

2.6.2 CRC

2.6.3 Ekstraktor Toeplitz'a

2.6.4 AES-256

2.7 Metody badania źródeł entropii

2.7.1 ENT

2.7.2 NIST SP 800-22

2.7.3 NIST SP 800-90B

2.8 Moduły testowe

2.8.1 RCT

2.8.2 APT

2.9 Sprzęt

2.9.1 Digilent CoraZ7-07S

2.9.2 Digilent Nexys 4 DDR

Rozdział 3

Testy bazowe

W niniejszym rozdziale przeanalizowano wpływ zastosowania korekcji źródła oraz przetwarzania końcowego na poziom generowanej entropii w domyślnej implementacji neoTRNG. Rozważane elementy architektury zostały przedstawione na rysunku 2.3 jako moduły „de-biasing” oraz „sampling”. W implementacji bazowej zastosowano korektor von Neumanna oraz mechanizm kompresji entropii oparty na CRC. W ramach przeprowadzonych testów porównano cztery warianty konfiguracji generatora:

1. moduł z korektorem von Neumanna oraz CRC (domyślny),
2. moduł z korektorem von Neumanna bez CRC,
3. moduł bez korektora z aktywnym CRC,
4. moduł bez korektora i bez CRC.

3.1 Modyfikacja

Testy przeprowadzono na zmodyfikowanym module neoTRNG. Moduł rozszerzono o dwa parametry generyczne *ENABLE_VON_NEUMANN* oraz *ENABLE_CRC*. Zmodyfikowany plik źródłowy znajduje się w załączniku A neoTRNG_mod_1.vhd.

Każdy z przypadków testowych uruchomiono na sprzęcie, a uzyskane wyniki zapisano w odpowiednich plikach.

1. ch3/VN_CRC.bin
2. ch3/nVN_CRC.bin
3. ch3/VN_nCRC.bin
4. ch3/nVN_nCRC.bin

3.2 NIST SP 800-22

TABELA 3.1: NIST SP 800-22 Statistical Test Results

Statistical Test	Pass Ratio
Frequency	100/100
BlockFrequency	98/100
CumulativeSums	100/100
CumulativeSums	99/100
Runs	99/100
LongestRun	99/100
Rank	98/100
FFT	97/100
NonOverlappingTemplate (148 tests)	14717/14800
OverlappingTemplate	99/100
Universal	99/100
ApproximateEntropy	100/100
RandomExcursions (8 tests)	495/504
RandomExcursionsVariant (18 tests)	1121/1134
Serial (2 tests)	199/200
LinearComplexity	100/100

Rozdział 4

Analiza rozmieszczenia

Rozdział 5

Faza algorytmiczna i optymalizacja ekstrakcji

Rozdział 6

Faza odporności i monitorowanie online

Rozdział 7

neoTRNG na FPGA dla RPI

Rozdział 8

Zakończenie

Literatura

- [1] Fourmilab. Ent: A pseudorandom number sequence test program.
- [2] Andrew Rukhin, Juan Soto, James Nechvatal, Miles Smid, Elaine Barker, Stefan Leigh, Mark Levenson, Mark Vangel, David Banks, N. Heckert, James Dray, San Vo, and Lawrence Bassham. Nist sp 800-22: A statistical test suite for random and pseudorandom number generators for cryptographic applications. *NIST Information Technology Laboratory*, 2010.
- [3] Lakshmi Sreekumar and P. Ramesh. Selection of an optimum entropy source design for a true random number generator. *Procedia Technology*, 2016.
- [4] stnolting. neotrng: A true random number generator for fpgas, 2016.
- [5] Meltem Sönmez Turan, Elaine Barker, John Kelsey, Kerry McKay, Mary Baish, and Michael Boyle. Nist sp 800-90b: Recommendation for the entropy sources used for random bit generation. *NIST Information Technology Laboratory*, 2018.

Dodatek A

Kod źródłowy

A.1 neoTRNG_mod_1.vhd

```
1  library ieee;
2      use ieee.std_logic_1164.all;
3      use ieee.numeric_std.all;
4
5  entity neoTRNG is
6      generic (
7          NUM_CELLS          : natural range 1 to 255;
8          NUM_INV_START      : natural range 3 to 4095;
9          NUM_RAW_BITS       : natural range 1 to 4096;
10         SIM_MODE           : boolean;
11         ENABLE_VON_NEUMANN : boolean := true;
12         ENABLE_CRC         : boolean := true
13     );
14     port (
15         clk_i      : in  std_ulogic;
16         rstn_i     : in  std_ulogic;
17         enable_i   : in  std_ulogic;
18         valid_o    : out std_ulogic;
19         data_o     : out std_ulogic_vector(7 downto 0)
20     );
21 end entity;
22
23 architecture neoTRNG_rtl of neoTRNG is
24
25     function clog2_f(x : natural) return natural is
26     begin
27         for i in 0 to natural'high loop
28             if (2 ** i >= x) then
29                 return i;
30             end if;
31         end loop;
32         return 0;
```

```

33     end function;
34
35     component neoTRNG_cell
36         generic (
37             NUM_INV    : natural;
38             SIM_MODE    : boolean
39         );
40         port (
41             clk_i      : in  std_ulogic;
42             rstn_i     : in  std_ulogic;
43             en_i       : in  std_ulogic;
44             en_o       : out std_ulogic;
45             rnd_o      : out std_ulogic
46         );
47     end component;
48
49     signal cell_en_in  : std_ulogic_vector(NUM_CELLS - 1 downto 0);
50     signal cell_en_out : std_ulogic_vector(NUM_CELLS - 1 downto 0);
51     signal cell_rnd    : std_ulogic_vector(NUM_CELLS - 1 downto 0);
52     signal cell_sum    : std_ulogic;
53
54     signal debias_sreg : std_ulogic_vector(1 downto 0);
55     signal debias_state : std_ulogic;
56     signal debias_valid : std_ulogic;
57     signal debias_data  : std_ulogic;
58
59     -- raw bitstream / bypass von neumann --
60     signal raw_valid : std_ulogic;
61     signal raw_data  : std_ulogic;
62
63     -- selected stream after optional debiasing
64     signal selected_valid : std_ulogic;
65     signal selected_data  : std_ulogic;
66
67     signal sample_en : std_ulogic;
68     signal sample_sreg : std_ulogic_vector(7 downto 0);
69     signal sample_cnt : std_ulogic_vector(clog2_f(NUM_RAW_BITS) downto 0);
70
71     constant poly_c : std_ulogic_vector(7 downto 0) := "00000111";
72
73     begin
74
75     assert false
76         report "[neoTRNG] The neoTRNG (v3.4) - A Tiny and Platform-Independent True
77             ↪ Random Number Generator, " & "github.com/stnolting/neoTRNG"
78             severity note;

```

```

78
79  assert (NUM_INV_START mod 2) /= 0
80      report "[neoTRNG] Number of inverters in first cell [NUM_INV_START] has to be
    ↪  odd!"
81      severity error;
82
83  assert 2 ** clog2_f(NUM_RAW_BITS) = NUM_RAW_BITS
84      report "[neoTRNG] Number of pre-processed raw bits [NUM_RAW_BITS] has to be a
    ↪  power of 2!"
85      severity error;
86
87  assert not SIM_MODE
88      report "[neoTRNG] Simulation-mode enabled (NO TRUE/PHYSICAL RANDOM)!"
89      severity warning;
90
91  entropy_cell_gen: for i in 0 to NUM_CELLS - 1 generate
92      neoTRNG_cell_inst: neoTRNG_cell
93          generic map (
94              NUM_INV => NUM_INV_START + (2 * i),
95              SIM_MODE => SIM_MODE
96          )
97          port map (
98              clk_i => clk_i,
99              rstn_i => rstn_i,
100             en_i => cell_en_in(i),
101             en_o => cell_en_out(i),
102             rnd_o => cell_rnd(i)
103         );
104  end generate;
105
106  cell_en_in <= cell_en_out(NUM_CELLS - 2 downto 0) & sample_en;
107
108  combine: process (cell_rnd)
109      variable tmp_v : std_ulogic;
110  begin
111      tmp_v := '0';
112      for i in 0 to NUM_CELLS - 1 loop
113          tmp_v := tmp_v xor cell_rnd(i);
114      end loop;
115      cell_sum <= tmp_v;
116  end process;
117
118  -- John von Neumann Randomness Extractor (De-Biasing)
    ↪ -----
119  -----
    ↪ -----

```

```

120     debiasing: process (rstn_i, clk_i)
121     begin
122         if (rstn_i = '0') then
123             debias_sreg <= (others => '0');
124             debias_state <= '0';
125         elsif rising_edge(clk_i) then
126
127             if ENABLE_VON_NEUMANN then
128                 debias_sreg <= debias_sreg(0) & cell_sum;
129                 -- start operation when last cell is enabled and process in every second
130                 -- cycle --
131                 debias_state <= (not debias_state) and cell_en_out(cell_en_out'left);
132             else
133                 -- VON NEUMANN DISABLED, ignore debias_state --
134                 debias_state <= '0';
135             end if;
136
137         end if;
138     end process;
139
140     debias_valid <= debias_state and (debias_sreg(1) xor debias_sreg(0));
141     debias_data  <= debias_sreg(0);
142
143     -- bypassed signals --
144     raw_valid <= cell_en_out(cell_en_out'left);
145     raw_data  <= cell_sum;
146
147     selected_valid <= debias_valid when ENABLE_VON_NEUMANN else raw_valid;
148     selected_data  <= debias_data  when ENABLE_VON_NEUMANN else raw_data;
149
150     -- Sampling Control
151     -- -----
152     -- -----
153     -- -----
154     sampling_control: process (rstn_i, clk_i)
155     begin
156         if (rstn_i = '0') then
157             sample_en <= '0';
158             sample_cnt <= (others => '0');
159             sample_sreg <= (others => '0');
160
161         elsif rising_edge(clk_i) then
162             sample_en <= enable_i;
163
164             if ENABLE_CRC then

```

```

163
164     -- CRC ENABLED --
165     if (sample_en = '0') or (sample_cnt(sample_cnt'left) = '1') then -- start
        ↪ new iteration
166         sample_cnt <= (others => '0');
167         sample_sreg <= (others => '0');
168     elsif (selected_valid = '1') then -- valid raw random bit
169         sample_cnt <= std_ulogic_vector(unsigned(sample_cnt) + 1);
170         -- CRC-style sampling shift-register to mix random stream --
171         if ((sample_sreg(sample_sreg'left) xor selected_data) = '1') then --
            ↪ feedback bit
172             sample_sreg <= (sample_sreg(sample_sreg'left - 1 downto 0) & '0')
                ↪ xor poly_c;
173         else
174             sample_sreg <= (sample_sreg(sample_sreg'left - 1 downto 0) & '0');
175         end if;
176     end if;
177
178     else
179         -- CRC DISABLED --
180         -- no counter usage at all
181         sample_cnt <= (others => '0');
182
183         if (sample_en = '1') and (selected_valid = '1') then
184             sample_sreg <= sample_sreg(sample_sreg'left-1 downto 0) &
                ↪ selected_data;
185         end if;
186
187     end if;
188 end if;
189 end process;
190
191 data_o <= sample_sreg;
192 valid_o <= sample_cnt(sample_cnt'left) when ENABLE_CRC else selected_valid;
193
194 end architecture;
195
196 library ieee;
197 use ieee.std_logic_1164.all;
198
199 entity neoTRNG_cell is
200     generic (
201         NUM_INV : natural;
202         SIM_MODE : boolean
203     );
204     port (

```

```

205     clk_i  : in  std_ulogic;
206     rstn_i : in  std_ulogic;
207     en_i   : in  std_ulogic;
208     en_o   : out std_ulogic;
209     rnd_o  : out std_ulogic
210 );
211 end entity;
212
213 architecture neoTRNG_cell_rtl of neoTRNG_cell is
214
215     signal sreg      : std_ulogic_vector(NUM_INV - 1 downto 0);
216     signal latch     : std_ulogic_vector(NUM_INV - 1 downto 0);
217     signal inv_in    : std_ulogic_vector(NUM_INV - 1 downto 0);
218     signal inv_out   : std_ulogic_vector(NUM_INV - 1 downto 0);
219     signal sync      : std_ulogic_vector(1 downto 0);
220
221 begin
222
223     en_shift_reg: process (rstn_i, clk_i)
224     begin
225         if (rstn_i = '0') then
226             sreg <= (others => '0');
227         elsif rising_edge(clk_i) then
228             sreg <= sreg(sreg'left - 1 downto 0) & en_i;
229         end if;
230     end process;
231
232     en_o <= sreg(sreg'left);
233
234     ring_osc_gen: for i in 0 to NUM_INV - 1 generate
235
236         latch(i) <= '0'          when (en_i = '0')      else
237                     latch(i)    when (sreg(i) = '0')   else
238                     inv_out(i);
239
240         inverter_phy: if not SIM_MODE generate
241             inv_out(i) <= not inv_in(i);
242         end generate;
243
244         inverter_sim: if SIM_MODE generate
245             inverter_sim_ff: process (rstn_i, clk_i)
246             begin
247                 if (rstn_i = '0') then
248                     inv_out(i) <= '0';
249                 elsif rising_edge(clk_i) then
250                     inv_out(i) <= not inv_in(i);

```

```
251         end if;
252     end process;
253 end generate;
254
255 end generate;
256
257 inv_in <= latch(NUM_INV - 2 downto 0) & latch(NUM_INV - 1);
258
259 synchronizer: process (rstn_i, clk_i)
260 begin
261     if (rstn_i = '0') then
262         sync <= (others => '0');
263     elsif rising_edge(clk_i) then
264         sync <= sync(0) & latch(latch'left);
265     end if;
266 end process;
267
268 rnd_o <= sync(1);
269
270 end architecture;
271
```


Dodatek B

Raport

Pytania

1. Kwestia sprzętu: trzymać się swojego czy czekać na Nexys 4?
 - Obecnie pracuję na swoim. Zależnie od decyzji dr. Kropidłowskiego pozostanę przy swoim rozwiązaniu lub powtórzę testy na Nexys.
2. Jak dodać w tekście informację o licencji neoTRNG?
3. Czy można cytować README repozytorium GitHub?
4. Czy dodać spis fragmentów kodu tak jak spis rysunków i tabel?
5. Jak powiedzieć de-biasing / sampling po polsku?
 - W tekście użyłem: de-biasing → korekcja źródła / post-processing -> przetwarzanie końcowe / sampling → próbkowanie / CRC → kompresja entropii.
6. Ile MiB danych do wykonania testów? (zbierałem 10 MiB)
 - Będę zbierać po 100 MiB.
7. NIST SP 800-90B: używać testów IID czy non-IID?
 - Wykonywałem testy non-IID, ewentualnie będą do poprawy.
8. Kto jest autorem neoTRNG?
9. Jak zestawić dane z testów?
 - Wyniki testów zestawie jak w Pańskim artykule. Pozostaje kwestia rozkładów C — ustaliliśmy, że jeszcze się Pan nad tym zastanowi.
10. Publikacje naukowe do uwzględnienia w pracy – sugestie lub wymagania

Notatki

1. Najwięcej interwencji z Pańskiej strony będzie wymagał wstęp do pracy. Na razie przygotowałem wstępny zarys, jednak brakuje mu sprecyzowania najważniejszych kwestii oraz nadania tonu reszcie pracy.
2. W rozdziale teoretycznym wybrałem zagadnienia, które mogłyby być uwzględnione. Ewentualnie proszę o informację, czego brakuje, a co jest zbędne.

Dodatek C

Notatki

Do zrobienia

1. Dodaj załącznik B z kodem źródłowym
2. Przerób wykresy .png na .svg
3. Popraw schematy .svg z Vivado



© 2026 Szymon Krzysztof Gogulski

Instytut Informatyki, Wydział Informatyki i Telekomunikacji
Politechnika Poznańska

Skład przy użyciu systemu $\text{\LaTeX}$ na platformie Overleaf.